C PROGRAMMING — ASSIGNMENT 1

Task 5: Header Files and Conditional Compilation

Marks: 3 | Total assignment: 50 marks

Explanation

A header contains shared declarations and is protected by include guards to prevent repeated inclusion within one translation unit. Use #include <stdio.h> for standard library headers and #include "billing.h" for a project header; quoted includes normally search the current source directory first. #ifdef DEBUG includes debugging statements only when the macro is defined, for example gcc -DDEBUG. A run-time if (debug) retains the branch and debug strings in the executable, while conditional compilation can omit them entirely in a non-debug build.

Suggested structure

Keep declarations and shared constants in billing.h, implementations in billing.c, and main() in main.c. Compile and link both source files together. Debugging output should go to stderr so normal bill output stays parseable.

C Source Code / Implementation

```
/* billing.h */
#ifndef BILLING_H
#define BILLING_H
#define FIXED_CHARGE_PAISE 5000LL
long long compute_bill(int units);
#endif

/* billing.c */
#include "billing.h"
#ifdef DEBUG
#include <stdio.h>
#define DEBUG_PRINT(...) fprintf(stderr, __VA_ARGS__)
#else
#define DEBUG_PRINT(...) ((void)0)
#endif

long long compute_bill(int units) {
    long long energy = 0;
    if (units > 200) energy += (long long)(units - 200) * 450;
    if (units > 100) {
        int second = (units < 200 ? units : 200) - 100;
        energy += (long long)second * 225;
    }
    DEBUG_PRINT("units=%d energy=%lld\n", units, energy);
    return FIXED_CHARGE_PAISE + energy;
}

/* main.c */
#include <stdio.h>
#include "billing.h"
int main(void) {
    printf("Bill for 150 units: Rs %lld.%02lld\n",
           compute_bill(150) / 100, compute_bill(150) % 100);
    return 0;
}
/* Build: gcc -Wall -Wextra main.c billing.c -o billing
   Debug: gcc -Wall -Wextra -DDEBUG main.c billing.c -o billing-debug */
```

Submission note

Compile and run the code on your own system, save actual terminal output/screenshots, and include them in your report. Review the code and be prepared to explain it during the viva. Any machine-specific measurements must come from your own execution.